

Comparison performance of Traditional and Machine Learning Approaches Using the CoNLL-2003 Dataset

ANTHONY BUSH CASTRO 20-CCS-281

Supervised By: Dr. Manzu Gerald S. Kenyi

**Department of Computer Science
School of Computer Science and Information Technology,
University of Juba**



**Graduation Thesis submitted in partial fulfillment of the requirements leading to the
award of Bachelor of Science (Hons) in Information Technology**

March 2025

Abstract

This research proposal presents a study aimed at comparing traditional and machine learning approaches to Named Entity Recognition (NER) using the CoNLL-2003 dataset. The study will evaluate traditional methods, including gazette-based approaches and statistical models like HMMs, alongside machine learning techniques such as SVM and CRF. These models will be assessed using standard evaluation metrics; precision, recall, and F1-score to comprehensively measure their effectiveness. By analysing the strengths and weaknesses of each approach, this research seeks to provide insights into relative advantages of these methodologies and their applicability across various domains. The findings are expected to contribute to the development of more robust and efficient NER systems, enhancing the broader field of Natural Language Processing (NLP) by guiding practitioners in selecting the most appropriate techniques for their specific needs.

Chapter 1: Introduction

1.1 Background

As a convergence of computer science, linguistic theory, and artificial intelligence, Natural Language Processing (NLP) represents a transformative technological domain focused on enabling computational systems to analyze, understand, and generate human language ([Gonzalez-Gomez, L.J., Hernandez-Munoz, S.M., Borja, A., Azofeifa, J.D., Noguez, J. and Caratozzolo, P., 2024](#)). This interdisciplinary field addresses the fundamental challenge of creating meaningful interactions between humans and machines by developing computer algorithms capable of processing both textual and spoken communication. Contemporary NLP systems utilize advanced machine learning architectures, including deep neural networks, which demonstrate remarkable proficiency in linguistic pattern recognition and semantic analysis when trained on large-scale textual corpora.

The practical implementations of NLP technologies span multiple critical domains. Sentiment interpretation, a prominent application, employs computational methods to detect and classify emotional tones and subjective content within digital text. Organizations leverage these capabilities for diverse purposes ranging from evaluating consumer feedback and brand perception to analyzing sociopolitical discourse and identifying harmful online content. Equally significant is automated language translation, which eliminates linguistic barriers by dynamically converting textual content between languages while preserving contextual meaning. This capability has revolutionized global information exchange and cross-cultural communication.

Foundational to NLP operations are text analysis techniques that systematically transform unstructured language data into machine-interpretable formats. The analytical pipeline typically incorporates lexical decomposition (tokenization), grammatical classification (POS tagging), and semantic entity identification ([Wang, Y., Sun, Y., Ma, Z., Gao, L., Xu, Y. and Sun, T., 2020](#)).

Named Entity Recognition, in particular, serves as a critical information extraction mechanism by detecting and categorizing specific data elements within text corpora ([MCCOY, A., WRIGHT, A., EYSENBACH, G., MALIN, B., PATTERSON, E., XU, H. & SITIG, D. 2013](#)). The subsequent discussion will concentrate on examining NER's operational

frameworks, current limitations, and potential applications within modern computational linguistics.

1.2 Problem Statement

As Named Entity Recognition (NER) is an essential component of information extraction from structured text in Natural Language Processing (NLP), it plays a critical role in identifying and classifying spans of text referred to as entities, such as proper nouns ([Yu, J., Bohnet, B. and Poesio, M., 2020](#)). NER involves the recognition and categorization of named entities within a text into predefined categories, including organisations, locations, dates, persons, and other domain-specific terms. Despite significant advances in NLP technologies, the task of identifying and classifying these entities remain challenging due to several factors, which include, but are not limited to:

Lexical Ambiguity and context independency

Named Entities often exhibit a wide variety of forms including abbreviations, acronyms, and synonyms. A single entity can have multiple names or references, making it difficult to accurately identify the intended meaning without context. The challenge is further compounded by the fact that words and phrases can have different meanings depending on their usage in a sentence. For instance, in South Sudan, the English words “Anger” and “But” are used as personal names in the Dinka community. When these words appear in written text without pronunciation cues, they could easily lead to confusion as they also represent common English words with distinct meanings. Similarly, the “Apple” could refer to either a fruit or a renowned technology company, depending on the context in which it is used. Accurately disambiguating such entities requires sophisticated models capable of understanding and interpreting context.

Domain-specific challenges

NER models often face difficulties when dealing with specialised technologies that have unique meanings in different domains. A model trained to perform well in one domain, such as finance, may struggle when applied to data from another domain, like healthcare, where terms and entity relationships differ significantly. For example, the term “lead” might refer to a market leader in a business context but denotes a toxic metal in an environmental context. Adapting NER models to handle diverse domains requires domain-specific training data and often involves developing custom rules or models tailored to particular fields.

Limited Annotated Data

The performance and accuracy of NER models are heavily dependent on the availability of high-quality annotated datasets. These datasets are crucial for training models to recognize and classify entities effectively. However, creating annotated datasets is a labour-intensive process, particularly of less commonly spoken languages or specialised domains.

Consequently, many languages and fields suffer from a lack of annotated data, making it challenging to develop robust NER models for these areas. This data scarcity issue is particularly pronounced in languages with limited digital resources or those that are spoken by smaller populations, where obtaining annotated datasets may not be feasible. Efforts to expand annotated datasets across various languages and domains are essential to achieving comprehensive NER capabilities globally.

Comparative Analysis of CRF and CRF for NER: While CRF is widely used in structured sequence labeling, RF is known for its robustness in classification tasks. However, limited research has directly compared CRF and CRF for NER on standardized datasets.

This study addresses these challenges by conducting a comparative analysis of CRF and RF to determine their effectiveness in NER applications.

1.3 Objectives

1.3.1 Primary Objective

The main objective of this research is to evaluate the performance of CRF and RF models for Named Entity Recognition (NER) using the CoNLL-2003 dataset.

1.3.2 Specific Objectives

1. To implement and evaluate CRF and RF models for NER
2. To compare their performance using standard evaluation metrics (precision, recall, and F1-score).
3. To analyze the strengths and limitations of each approach in recognizing named entities.

1.4 Significance of the Study

This study contributes to NER research by providing a detailed comparison of CRF and RF, two prominent models used in structured text processing, the findings of this research are expected to:

1. Enhance the understanding of CRF and RF in NER applications
2. Provide insights into model selection for researchers and developers working on text classification tasks.
3. Support future advancements in machine learning-driven NER systems.
4. Offer a benchmark for evaluating NER models using the CoNLL-2003 dataset.

By addressing existing challenges and gaps, this study aims to advance the development of robust, scalable, and adaptable NER solutions for various domains.

1.5 Research Questions

This study aims to answer the following research questions:

1. How does the performance of CRF compare to RF in Named Entity Recognition?
2. What are the advantages and limitations of CRF and RF for NER tasks?
3. Which model demonstrates better generalization across different entity types in the CoNLL-2003 dataset?
4. What insights can be drawn from the comparative analysis to guide future research in NER?

1.6 Scope

This study focuses on Named Entity Recognition (NER) using the CoNLL-2003 dataset

It evaluates two machine learning models (CRF and RF)

The research is limited to English-language text, as provided in the dataset

Chapter Two: Literature Review

2.1 Introduction

Named Entity Recognition (NER) is a core task in Natural Language Processing (NLP) that involves identifying and classifying named entities within text into predefined categories such as persons, organisations, and locations ([Zirikly, A. and Diab, M., 2014](#)). Over time, various approaches have been developed for NER, ranging from ruled-based systems to machine learning and, more recently, deep learning techniques.

This chapter reviews existing literature on NER, highlighting traditional and modern approaches, their strengths and limitations, and recent advancements, particularly in Conditional Random Fields (CRFs) and Random Forest (RF) models.

2.2 Traditional Approaches to NER

2.2.1 Ruled-Based methods

Early NER systems often rely on rule-based and statistical methods. Rule-based systems used handcrafted linguistic rules and typically language-specific, requiring significant effort to adapt to new languages or domains ([Magnini, B., Lavelli, A. and Magnolini, S., 2020](#)).

Statistical methods, including Hidden Markov Models (HMMs) and Condition Random Fields (CRFs), have been widely used in NER due to their ability to model sequential data and capture dependencies between words (Magnini et al., 2020). CRFs, in particular, have shown strong performance in various NER tasks by considering the context of words and the dependencies between entity labels.

Limitations of Rule-Based methods

- **High maintenance:** Rules must be manually updated for new datasets.
- **Poor generalization:** Struggle with unseen entities and different domains.
- **Lack of context awareness:** Can not differentiate entities with ambiguous meanings (e.g., “Apple” as a fruit vs a company).

2.3 Machine Learning Approaches to NER

Machine learning techniques have significantly advanced NER by leveraging labelled data to train models that generalise better across languages and domains. Support vector machines (SVMs) and machine entropy models are among the traditional machine learning algorithms employed for NER ([Magnini, B., Lavelli, A. and Magnolini, S., 2020](#)). These models rely heavily on feature engineering, where domain-specific knowledge is used to design features that capture relevant information for entity recognition. The advent of deep learning has recognized NER by enabling the automatic extraction of features from raw text data.

Recurrent Neural Network (RNNs), Long Short-Term Memory (LSTM) networks, and Convolutional Neural networks (CNNs) are commonly used architectures in deep learning-based NER models ([Jiang, L. and Bolshakova, E., 2019](#)). These models can learn complex patterns in data without the need for extensive feature engineering. Recent advancements include the use of pre-trained language models like BERT (Bidirectional Encoder Representations from Transformers), which have set new state-of-the-art performance benchmarks in NER ([Magnini, B., Lavello, A. and Magnolini, S., 2020](#)).

2.3.1 Conditional Random Fields (CRF)

CRF is a probabilistic graphical model widely used for sequence labeling tasks like NER. Unlike earlier models such as Hidden Markov Models (HMMs), CRFs do not assume independence between input features, making them more effective for NLP tasks ([Lafferty, J., McCallum, A. and Pereira, F., 2001](#)).

Features of CRF for NER

- Captures dependencies between adjacent words (e.g., “New” is likely followed by “York” as a location).
- Uses feature functions to model complex relationships (e.g., capitalization, pos tags).
- Applies global normalization, avoiding bias in labeling sequences.

Limitations of CRF

- Computationally expensive, training requires large datasets
- Features engineering dependency, performance relies on well-designed input features.

2.3.2 Random Forest (RF)

Random Forest (RF) is an ensemble learning method that builds multiple decision trees and aggregates their predictions to improve classification performance ([Breiman, L., 2001](#)). Although RF is commonly used for structured classification tasks, it has been adapted for NER for extracting features from text data.

Features of RF for NER

- Handling high-dimensional data: Can process word embeddings, POS tags, and contextual features.
- Reduces overfitting: By averaging multiple decision trees, RF provides better generalization.
- Interpretable model: Feature importance score and reveal which features contribute most to classification.

Limitations of RF

- Ignore sequential dependencies: unlike CRF, RF does not capture contextual relationships between words.
- Higher memory usage: training multiple decision trees requires significant computational resources.

2.4 Related Work

Several studies have explored the effectiveness of CRF and RF for NER, demonstrating their strengths in structured text processing.

2.4.1 CRF-Based NER System (Guo et al., 2020)

Guo et al. (2020) conducted a study on CRF-based NER models, focusing on financial text processing. They trained a CRF model using handcrafted features, such as:

- **Word shape** (uppercase, lowercase, title case).
- **Part-of-speech** (POS) tags.
- **Contextual window** (neighbouring words).

The study found that CRF achieved an F1-score of 85.6%, outperforming rule-based methods. However, the authors noted that feature engineering was a major limitation, requiring expert knowledge for optimal performance.

2.4.2 RF for Text Classification (Zhou et al., 2021)

Zhou et al. (2021) applied Random Forest (RF) for NER in medical document classification. Instead of using CRF-style sequence modeling, RF was trained on:

Tf-IDF and word embeddings for feature representation.

POS tags and syntactic dependencies as additional features.

The study showed that RF performed well in structured datasets, with an accuracy of 87.3%. However, it struggled with context-sensitive entity recognition, as RF does not model word order.

These related works highlight the trade-offs between CRF and RF in NER tasks. CRF excels in sequence-dependent text processing, while RF is strong in structured classification tasks with diverse features.

2.5 Recent Advances in NER

With the rise of deep learning, newer NER models leverage architectures like Bidirectional Long Short-Term Memory (BiLSTM) with CRF and Transformer-based models (e.g., BERT, GPT) ([Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2019](#)). These models automatically learn features from large datasets, improving accuracy and reducing reliance on manual feature engineering.

Hybrid Model (BiLSTM-CRF)

Combine BiLSTM for feature extraction with CRF for structured prediction.

Improves generalization by learning context-aware representations.

More computationally expensive than the standalone CRF or RF.

Transformer-Based NER models

BERT and GPT use deep contextual embeddings for entity recognition.

State-of-the-art performance, outperforming traditional machine learning approaches

Requires large-scale training data and high computational resources

2.6 Research Gap

Despite advances in deep learning, models like **CRF** and **RF** remain widely used due to their interpretability, lower computational cost, and strong performance on structured datasets.

However, the following challenges remain:

- **CRF** relies on extensive feature engineering, requiring domain expertise.
- **RF** lacks sequence awareness, making it less effective for tasks needing contextual relationships.

There is limited comparative analysis between **CRF** and **RF** for **NER**, especially on standardized datasets like CoNLL-2003.

This study aims to fill this gap by comparing **CRF** and **RF** in NER tasks, evaluating their strengths and weaknesses using **precision**, **recall**, and **F1-score**.

Chapter Three: Methodology

3.1 Introduction

Named Entity Recognition (NER) stands as a pivotal task within the domain of information extraction and natural language processing ([Nadeau, D. and Sekine, S.,\(2007\)](#)). Its fundamental objective is to identify and categorize named entities present in unstructured text into predefined semantic classes, such as names of persons, organization, location, and miscellaneous entities ([Martin, J.H. and Jurafsky, D., \(2019\)](#)). The process involves not only recognizing these entities but also classifying them according to a specific typology. However, NER is not without its challenges, primarily stemming from ambiguities in segmentation, where multi-word entities like “New York” need to be correctly identified as a single location, and in tag assignment, where a term like “Apple” could refer to a fruit or a technology company depending on the context ([Yadav, V. and Bethard, S., 2019](#)). The capability to accurately identify these entities is crucial as it lays the groundwork for more advanced natural language understanding and information retrieval systems. Effective NER performance is increasingly important across a multitude of applications, including news analysis for tracking key entities, ([Ritter, A., Cherry, C. and Dolan, B., 2011](#)) customer service for understanding user inquiries, and biomedical text processing for extracting critical information from research literature. ([Wei, W., Zhang, X., Liu, X., Chen, W. and Wang, T., 2016](#))

The CoNLL-2003 dataset has emerged as a widely recognized benchmark for the task of language-independent named entity recognition ([Sang, T.K., 2003. EF and De Meulder](#)). This dataset, originating from the CoNLL-2003 shared task, provides annotated news articles both in English and German, although this analysis will primarily focus on the English portion. The dataset’s enduring popularity as a standard evaluation resource underscores its significance in the field, offering a consistent measure for comparing the effectiveness of various NER models over time. Nevertheless, it is important to acknowledge that the dataset’s creation, with data sourced from news articles in 1996, might introduce a discrepancy when evaluating contemporary models on more recent textual data, given the natural evolution of language and the emergence of new entities ([Ratinov, L. and Roth, D., 2009](#)).

In the realm of sequence labeling and classification, two prominent modeling techniques are Conditional Random Fields (CRF) and Random Forest (RF). Conditional Random Fields are a type of discriminative probabilistic graphical model that are particularly well-suited for tasks involving sequential data, such as NER ([Lafferty, J., McCallum, A. and Pereira, F.,](#)

[2001](#)). On the other hand, Random Forest is an ensemble learning method that operates by constructing a multitude of decision trees during training and then outputs the mode of the classes (for classification) or the mean prediction (for regression) of the individual trees ([Breiman, L., 2001](#)). . The primary aim of this research is to conduct a comparative analysis of the performance and inherent characteristics of CRF and RF models when applied to the task of Named Entity Recognition using the CoNLL-2003 dataset. Given the fundamental difference in the nature of these models, with CRF being inherently designed for sequence labeling while RF is primary classification algorithm, careful consideration must be given to how RF can effectively adapt to address the sequential nature of the NER problem ([Kumar, S., Stecher, G., Li, M., Knyaz, C. and Tamura, K., 2018](#))

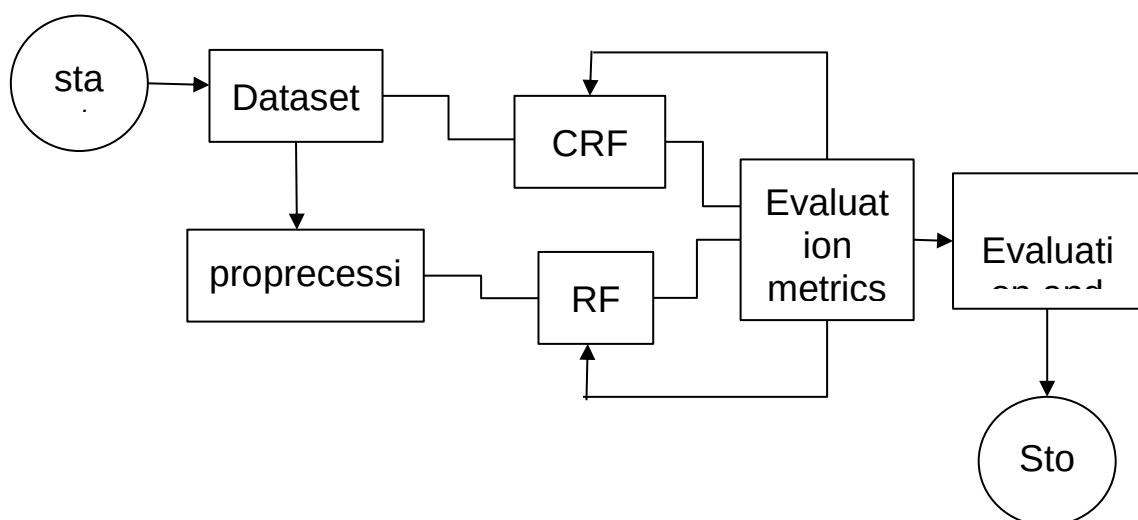
3.2 Research Design

Research design refers to the overall strategy or framework that outlines how a research study will be conducted. It encompasses the methods and producers for collecting, measuring, and analyzing data to answer specific research questions of hypotheses. ([Trochim, William M.K. Research Methods Knowledge Base. 2006, \(Web\).\)](#)

Below are the steps for the implementation of these methodology

1. **Dataset Selection:** using the CoNLL-2003 dataset, a benchmark dataset for NER.
2. **Data preprocessing:** Tokenization, sentence segmentation, and feature extraction
3. **Model Implementation:** Training and testing CRF and RF models
4. **Evaluation:** measuring model performance using standard evaluation metrics.

The figure below shows the implementation of the methodology.



3.2 The CoNLL-2003 dataset

The CoNLL-2003 dataset comprises news articles in both English and German languages, with the English data being the primary focus for many NER studies ([Sang, T.K., 2003. EF and De Meulder](#)). The dataset is structured such that each instance is presented across four columns, separated by a single space, within the data files designated for training, validation, and testing. The first column contains the word or token itself, representing the basic unit of analysis. The second column provides the part-of-speech (POS) tag for each word, indicating its grammatical role in the sentence ([Martin, J.H. and Jurafsky, D., 2019](#)). Following this, the third column contains a syntactic chunk tag, which identifies the constituent parts of the sentence, such as noun phrases or verb phrases. Finally, the fourth column represents the Named Entity tag, which labels each word as belonging to a specific named entity category or as not being part of any entity. Each word in a sentence occupies a separate line, and sentence boundaries are clearly demarcated by empty lines.

The Named Entity tag within the CoNLL-2003 dataset adheres to the IOB2 tagging scheme ([Sang, E..F. and Veenstra, J., 1999](#)). In this scheme, a tag prefixed with “B”-(Begin) indicates the start of a named entity of a particular type. A tag prefixed with “I”- (Inside) signifies that the word is within a named entity of that type, and it follows a “B”- or another “I”- tag of the same type. Words that do not constitute any name entity are tagged with “O”- (Outside). The dataset specifically focuses on four primary types of named entities: Persons (PER), Locations (LOC), Organizations (ORG), and Miscellaneous entities (MISC) that do not fall into the other three categories. It is worth noting that the original CoNLL-2003 data files might have included “DOCSTART” lines to delineate the beginning of new documents, but these are often removed in processed versions of the dataset used on various implementations. The presence of POS and chunk tags alongside the NER tags offers an opportunity for feature engineering, where these linguistic features can be leveraged to potentially enhance the performance of NER models ([Ratinov, L. and Roth, D., 2009](#)). For instance, the grammatical context provided by POS tags and phrasal boundaries indicated by chunks tags could serve as valuable cues for identifying and classifying named entities.

The English CoNLL-2003 dataset is conventionally partitioned into three distinct subsets to facilitate the training, evaluation, and deployment of NER models. The **training set** serves as the primary data source for training the machine learning models. This set comprises approximately 14,041 sentences or 203, 621 tokens across 946 news articles. The **validation set**, also known as the development set, plays a crucial role in tuning the hyperparameters of the models and evaluating their performance during the training process to mitigate the risk of overfitting. It consists of roughly 3, 250 sentences or 51, 362 tokens derived from 216 articles. Finally, the **test set**, sometimes referred to as the prediction set, is reserved for the final assessment of the trained model’s generation capability on data that the model has never encountered before. This set includes approximately 3, 453 sentences or 46, 435 tokens from 231 articles. The data for the English portion of the CoNLL-2003 dataset was extracted from Reuters corpus, spanning news stories published between August 1996 and August 1997. Notably, the test set specifically contains articles from December 1996. The temporal separation between the training and validation data (primarily August 1996) and the test data (December 1996) is a deliberate design choice. This ensures that the model’s evaluation is conducted on temporally independent data, providing a more reliable indication of its ability to generalize to unseen instances. Furthermore, the substantial difference in size between the training set and the validation and test sets underscores the importance of having a sufficiently large training corpus to enable robust learning of patterns and relationships with the data ([Sang, T.K., 2003. EF and De Meulder](#)).

Table 1: CoNLL-2003 English Dataset Statistics

Split	Number of Articles	Number of Sentences	Number of Tokens
Training	946	14,987	203,621
Validation	216	3,466	51,362
Test	231	3,684	46,435

3.3 Models

3.3.1 Conditional Random Fields (CRF)

Conditional RandomFields (CRFs) are recognized as powerful discriminative undirected probabilistic graphical models, particularly adept at handling structured prediction tasks, with sequence labeling being a primary application. Unlike generative models that aim to model the joint probability distribution of both observations and labels, CRFs directly model conditional probability of a sequence of labels given an input sequence of observations, denoted by $P(y/x)$ ([Lafferty, J., McCallum, A. and Pereira, F., 2001](#)). For Natural Language Processing (NLP) tasks as Named Entity Recognition (NER), linear-chain CRFs are frequently employed. In this configuration, the underlying structure is a sequence of words, and the prediction of a label for each word is influenced by the labels of its neighbouring words in the sequence.

CRFs offer several advantages over other sequence labeling models like Hidden Markov Models (HMMs) and Maximum Entropy Markov Models (MEMMs). One key benefit is that CRFs relax the stringent independence assumptions that HMMs require, allowing for a richer and more flexible representation of features. This means that CRFs can effectively incorporate a wide array of potentially overlapping and non-independent features extracted from the observation sequence. Furthermore, CRFS addresses the label bias problem, a known limitation of MEMMs, where transitions from a state compete only against each other, potentially leading to suboptimal predictions. The ability of CRFs to handle arbitrary, non-independent features makes them particularly well-suited for NER, where the classification of a word as named entity often depends not only on its own characteristics but also on the context provided by surrounding words and various linguistic features associated with the word itself, such as its form, capitalization, and part-of-speech ([Sha, F. and Pereira, F., 2003](#)). The discriminative nature of CRFs, which focuses on directly modeling the conditional probability $P(y/x)$, makes them highly effective for classification tasks like NER, where the primary objective is to accurately predict the sequence of labels corresponding to the input text.

The mathematical formulation of a linear-chain CRF defines the conditional probability distribution for sequence of labels given an input sequence as follows:

$$P(y/x) = (1/z(x)) * \exp(\sum_{t=1}^N \{T\} \\ \sum_{k=1}^N \{k\} \lambda_k * f_k(y_{t-1}, y_t, x, t))$$

Here, $y = (y_1, y_2, \dots, y_T)$ represents the sequence of labels for the input sequence x of length T . The term $f_k(y_{t-1}, y_t, x, t)$ denotes the set of feature functions, where each function evaluates a specific characteristic of the input sequence at position t , considering the current label y_t and previous label y_{t-1} . These feature functions can be binary, indicating the presence or absence of a particular feature, or real-valued, representing the degree to which the feature is present.

Examples of such features in the context of NER include the identity and capitalization of the word (lexical features), its part-of-speech tag (syntactic feature), and the words surrounding it (contextual features) ([Sha, F. and Pereira, F., 2003](#))

The parameters λ_k are the weights associated with each feature function, indicating their importance in determining the probability of a label sequence. These weights are learned during the training phase by maximising the conditional likelihood of the labelled training data.

The term $Z(x)$ is the normalization constant, also known as the partition function, which ensures that the sum of probabilities over all possible label sequences for a given input sequence x equals 1.

It is calculated as:

$$Z(x) = \sum_{y'} \exp\left(\sum_{t=1}^T \sum_{k=1}^K \lambda_k f(y'_{t-1}, y'_t, x, t)\right)$$

Where the summation is over all possible label sequences y' . The primary goal when applying CRFs to NER is to find the label sequence y^* that yields the highest conditional probability for a given input sentence x , effectively identifying and classifying the named entities within text.

The design of effective feature functions is paramount in achieving high performance with CRF models for NER, as these functions allow the model to leverage a wide range of linguistic and contextual cues that are indicative of named entities ([Lafferty, J., McCallum, A. and Pereira, F., 2001](#)).

3.3.2 Random Forest (RF)

Random Forest (RF) is a widely used ensemble learning method in machine learning that operates by constructing a multitude of decision trees during the training phase and then aggregating their predictions to arrive at a final output. This algorithm is versatile and can be applied to both classification problems, where the goal is to predict a categorical label, and

regression problems, where the aim is to predict a continuous numerical value. The strength of the Random Forest algorithm relies in several key mechanisms:

Bootstrap Sampling (Bagging):

Each individual decision tree within the forest is trained on training data, with replacement. This process, known as bootstrap sampling of bagging, ensures that each tree learns from a slightly different perspective of the data, variance of the model and mitigates the risk of overfitting ([Breiman, L., 2001](#)).

Random Feature Selection:

When each tree is building its nodes, at each split point, the algorithm considers only a random subset of the available features. This randomness in feature selection further decorrelates the trees in the forest, making the ensemble more robust and less prone to being dominated by a single strong predictor ([Liaw, A. and Wiener, M., 2002](#)).

Growing Trees:

Typically, in a standard Random Forest, each decision tree is grown to its maximum possible depth without any pruning. This allows the individual trees to be as complex as they can be, capturing intricate patterns in their respective bootstrap samples

Final Prediction:

Once the forest of the trees is trained, for a classification task, the final prediction for a new data point is determined by taking a majority vote across the predictions made by all the individual trees in the Forest.

For a regression task, the final prediction is usually the average of the predictions from all the trees.

The final output of the Random Forest model can be summarized as follows:

For **classification**:

$$\text{Prediction} = \text{mode} (f_1(x), f_2(x), \dots, f_B(x))$$

Where $f_b(x)$ represents the prediction from the b^{th} tree

For **regression**:

$$\text{Prediction} = \frac{1}{B} \sum_{b=1}^B f_b(x)$$

Where B is the total number of trees in the forest

The Random Forest algorithm has several hyperparameters that can be tuned to optimize its performance for a specific task. These include the number of trees in the forest, the maximum

number of samples required to be at a leaf node, and the number of features to consider when looking for the best split at each node.

Random Forests offer several advantages, including high predictive accuracy, good robustness to outliers in the data, and reduced tendency to overfit the training data compared to single decision trees. Additionally, the algorithm provides a useful measure of feature importance, indicating which features in the dataset were most influential in making predictions ([Breiman, L., 2001](#)) .

Random Forest can also handle datasets with a mix of both numerical and categorical features without requiring extensive preprocessing. The underlying mechanism of combining multiple decision trees allows Random Forest to capture complex, non-linear relationships present in the data.

Applying Random Forest to sequence labeling tasks like Named Entity Recognition requires a specific approach, as RF is fundamentally a classification algorithm that typically operates on independent instances. To adapt RF for NER, the problem is often framed as classifying the NER tag for each individual word within a sequence. This involves carefully designing features for each word that can capture not only the properties of the word itself but also the relevant context from the surrounding words ([Liaw, A. and Wiener, M., 2002](#)) .

For instance, the feature for a given word could include the word itself, its POS tag, its chunk tag, and crucially, information about the words immediately preceding and succeeding it, such as the previous word, the next word, and their respective POS tags. Furthermore, numerical representations of words, such as word embeddings, can also be used as features. In this setup, each word in the CoNLL-2003 dataset is treated as separate instance, and the RF model is trained to classify it into one of the predefined NER tags (B-PER, I-PER, B-ORG, I-ORG, B-LOC, I-LOC, B-MISC, I-MISC, O). A critical point to note is that when using RF for NER in this manner, the direct sequential dependencies between the NER tags of consecutive words are not inherently modeled by the algorithm itself. Instead, these dependencies must be captured indirectly through the features that are engineered for each word.

The effectiveness of RF in this context heavily relies on the quality and comprehensiveness of these designed features, as they must encode the necessary sequential information that the algorithm does not inherently process when comparing RF to CRF for sequence labeling tasks like NER, a fundamental difference emerges. CRF is specifically designed to model the

dependencies between consecutive labels in a sequence, which uses a crucial aspect of NER where the tag of the word is often influenced by the tag of the word immediately before it.

In contrast,

While RF can be adapted for NER through feature engineering, it treats each token's labeling as an independent classification problem based on the provided features, potentially missing some of these direct sequential dependencies. Furthermore, CRFs, probabilistic framework provides a well-defined method for finding the optimal sequence, often using algorithms like the Viterbi algorithm.

RF, on the other hand, typically relies on majority voting or averaging of predictions from its constituent trees. Therefore, while RF can be employed for NER by carefully crafting features to represent the context, CRF's inherent design as a sequence labeling model makes it a more natural fit for tasks where the dependencies between labels in a sequence are significant. The explicit modeling of transitions between states in CRF allows it to better capture the grammatical and semantic constraints that are often characteristic of named entities.

3.4 Evaluation Metrics

To quantitatively assess the performance of Named Entity Recognition models, including CRF and RF, several standard evaluation metrics are commonly employed. These metrics provide insights into the model's accuracy and ability to correctly identify and classify named entities. The primary metrics used are Precision, Recall, and the F1-score ([Yacouby, R. and Axman, D., 2020](#)).

Precision

Precision measures the accuracy of the positive predictions made by the model. It answers the question: "of all the entities the model identified, how many were actually correct?" It is calculated as the ratio of correctly identified named entities (True positives) to the total number of entities identified by the model (True positive + False Positive).

The formula for Precision is:

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Recall

Recall, also known as sensitivity, measures the model's ability to find all the actual named entities present in the text. Answers the question: "of all the actual entities present, how many did the model correctly identify?" It is calculated as the ratio of correctly identified named

entities (True Positives) to the total number of actual named entities in the data (True Positives + False Negative). The formula for recall is:

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

The F1-Score

F1-Score is the harmonic mean of precision and recall. It provides a single, balanced metric that considers both the precision and recall of the model. The F1-score is particularly useful when there is an imbalance in the number of positive and negative instances in the dataset.

The formula for the F1-score is:

$$\text{F1-score} = 2 * \frac{(\text{Precision} * \text{Recall})}{\text{Precision} + \text{Recall}}$$

This can also be expressed in terms of True Positives (TP), False Positives, and False Negatives (FN) as:

$$\text{F1-score} = 2 * \text{TP} / (2 * \text{TP} + \text{FP} + \text{FN})$$

In these formulas:

True Positive (TP): Represents the number of times the models correctly identified a named entity that was actually present in the text.

False Positive (FP): Represents the number of times the model incorrectly identifies a word or phrase as a named entity when it was not.

False Negative (FN): Represents the number of times the model failed to identify a named entity that was actually present in the text.

True Negative (TN): While relevant in binary classification, True Negatives are less directly used in standard NER evaluation, accurate identification and classification of entity spans.

The F1-Score is often considered the most important metric for evaluating NER models because it provides a balance between the model's precision and its recall. In the context of NER, both making incorrect entity predictions (low precision) and missing actual entities (low recall) can be detrimental to the overall performance of the system. These evaluation metrics are typically calculated at two levels: at the entity level, where precision, recall, and the F1-score are computed for each specific type of named entity (e.g., Person, Location, Organization), and at the model level, where the metrics are aggregated across all entity types to provide an overall performance score for the model.

Analyzing the performance metrics for each individual entity type can reveal specific strengths and weaknesses of the model in recognizing different categories of named entities. For example, a model might be very good at identifying person names but less effective at recognizing organization names.

In addition to these metrics, a **Confusion Matrix** can be used as a visual tool to evaluate the performance of an NER model. A Confusion matrix is an $N \times N$ matrix, where N is the number of entity types (including the 'O' tag for non-entities). The matrix compares the actual labels of the entities in the test set with the labels predicted by the model.

The rows of the matrix typically represent the actual labels, while the columns represent labels (or vice versa).

The diagonal elements of the matrix show the number of correctly classified instances for each entity type (True Positive), while the off-diagonal elements indicate the instances where the model made incorrect predictions (False Positive and False Negatives). By examining the confusion matrix, it is possible to identify which entity types are frequently confused with each other by the model. This can provide valuable insights into the types of errors the model is making and can help in guiding further improvements to the model's architecture, features, or training data. For instance, if the matrix shows a high number of locations being misclassified as organizations, it might suggest that the features used to distinguish between these two entity types are not sufficiently discriminative.

3.5 Programming Language, Software, and Libraries Used in this research

The primary programming language of choice for implementing machine learning and Natural Language Processing (NLP) tasks is **Python**, largely due to its rich ecosystem of specialized libraries and frameworks. Python's syntax is designed to be both transparent and beginner-friendly, which, combined with a vast and active community of developers and researchers, makes it an ideal language for projects in the fields of NLP and machine learning. The extensive support facilitates rapid prototyping, experimentation, and collaboration among teams.

3.5.1 Libraries Used

3.5.2 Table 2 show the libraries used in the implementation of CRF and RF models

Libraries	purpose
-----------	---------

Pandas	Handling structured dataset (e.g., CoNLL-2003 dataset)
numpy	Numerical computations for feature extraction
Sklearn(scikit-learn)	Machine learning utilities (Random Forest, Evaluation metrics)
sklearn.ensemble.RandomForestClassifier	Random Forest model implementation
sklearn.feature_extraction.DictVectorizer	Converting features into numerical form
sklearn.metrics	Precision, Recall, and F1-score
sklearn.model.selection.train_test_split	Splitting dataset into training and test sets
sklearn.pipeline.pipeline	Creating model pipeline for structured processing
sklearn_crfsuite	CRF model implementation
Nltk (natural language toolkit)	Tokenization, part-of-speech (POS) tagging
spacy	Advanced NLP processing for feature extraction
Matplotlib and seaborn	Visualization of model performance

3.5.3 Code editors used

Jupyter Notebook/ JupyterLab: These provide an interactive, web-based environment where code can be written in cells, often used for data exploration, prototyping, and creating shareable documents that combine code, text, and visualizations.

Visual Studio Code (VS Code): This is a widely adopted and highly customizable code editor from microsoft, known for its excellent support for python through extensions, as well as its debugging capabilities and integration with Git.

3.6 Conclusion

This chapter detailed the methodology for evaluating CRF and RF models for Named Entity Recognition (NER) using the CoNLL-2003 dataset. It covered selection, preprocessing, feature extraction, model implementation, and evaluation metrics (precision, recall, and F1-score). CRF, a sequence-labeling model, was compared with RF, an ensemble classifier adapted for NER. Python libraries such as sklearn, NLTK, and spaCy were used for implementation. The next chapter presents the experimental results and performance analysis.

Results: